Design Exercise 1 — 02/06/25 - Sketching initial specs and design — Eng Notebook - Savanna Coffel

goals: need to be able to:

- create account
- log in to account
- list accounts via text wildcard - so if I needed to search for an account, I can
- send message to a recipient through the central server
- read messages: if someone is logged in, deliver immediately; else keep a queue
- delete messages
- delete accounts

MY ASSUMPTIONS

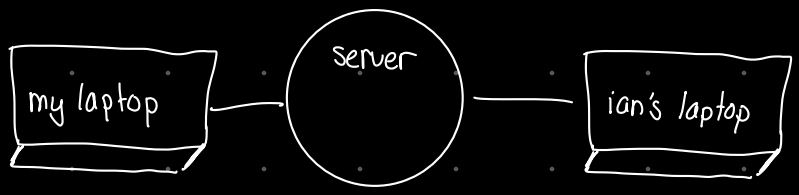
- all clients and the server are written in Python
- if a client sends a message to another client, that message goes through the central server, which then forwards it to the client
- our passwords have to be hashed. We will do this on the client side and only send hashed passwords over the wire
- clients cannot send messages to each other until they have established a connection with the server
- Store unread messages in a priority queue - likely array in Python with most recent message last in the queue to preserve order
- when we delete an account, if it has unread messages, we prompt if the user wants to read them. they can say yes or no
- we send over all messages as utf-8 encoded strings. the strings contain sender, reciever, message info in the form:

{ uuid | datetime | sender | reciever | message content }

↳ this is a dict that gets turned into a string → encoded in binary and sent over the network

↳ messages saved to internal database as the dict, so the server can do operations on it

Wire Protocol 02/07/25 - 02/08/25



→going to follow TCP/IP

→send encoded strings over the network under utf-8

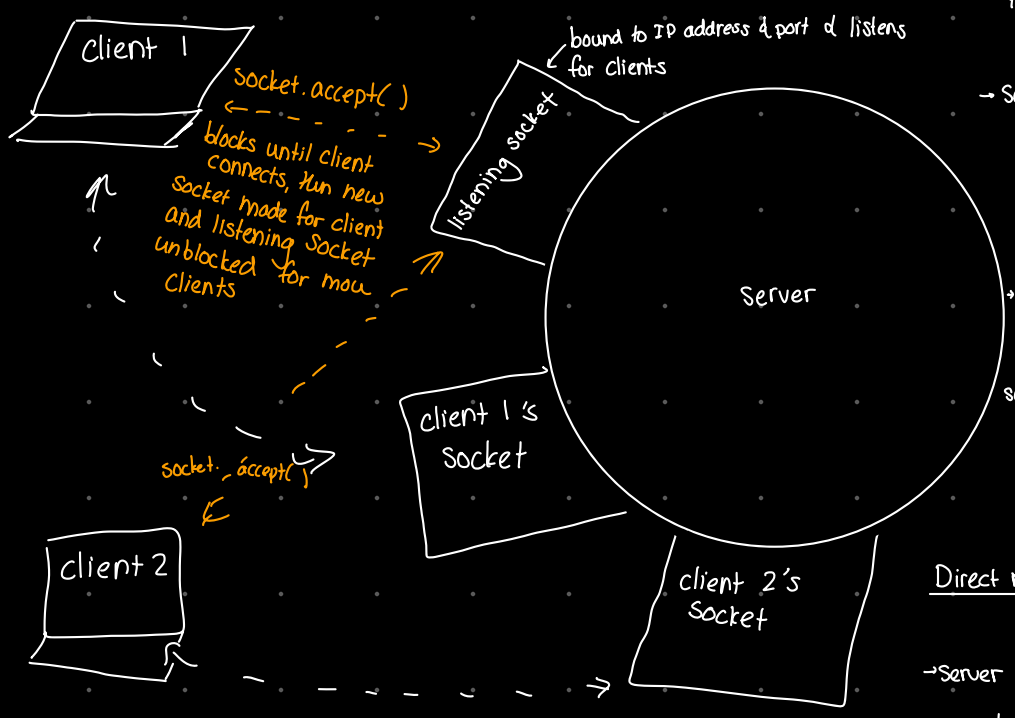
→ need to check multiple sockets at once to read/write to/from pipe w/ client

→ Select.select() checks multiple sockets at once

checks read, write, exception list of sockets

→listening socket set to socket.setblocking(False)

so listening socket continuously polls instead of blocks



-server will pass messages between clients by looping over select() calls

-chatlogs get updated w/ timestamps, user info every time a message is sent or recieved

Direct messaging system

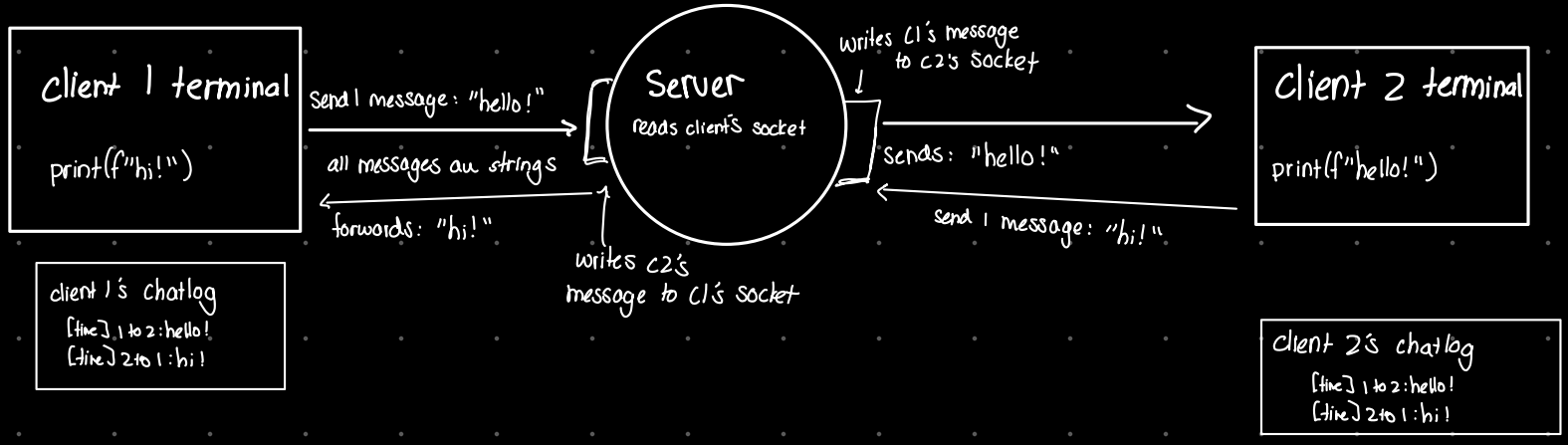
→server stores a map of usernames to client sockets

↳server routes to specific client

→client specifies which username to send the message to

→ server stores map of clients to sockets by:

{username, socket}, draws from reciever key in message to find the right socket



Wire Protocol: We should be able to switch between JSON mode & normal mode by flipping a boolean.

→ Our custom wire protocol sends dicts that are "stringified" into strings encoded as utf-8. Ex: "{uuid|datetime|sender|reciever|content}"

↳ strings get sent over the network as binary using utf-8 conversion

Creating a new account:

02/07/25 - Working on data management and networking

- when we log into an account, establish the client-server connection via accept() & other calls (easy)

- there are exactly as many sockets as there are currently logged-in users

- server maps: { username : assigned socket }

accounts = {
  uuid: { uuid: ___ | user: ___ | pass: ___ }
  uuid: { ... }
  uuid: { ... }
}

- Hash password client side

Offline messaging:

02/08/25: Messaging. Harder than I thought!

- if the desired client is offline, the server will store the messages in a dictionary with the same format as the other messages

pending_messages = { 'username 1': ["this is a message you'll read later!"]

'username 2': ["this is another, different message!"] }

- when user logs back in, parse dictionary for a key match - if one exists, prompt that client & display # of messages

if client wants to see it, deliver messages by writing to client's socket

Clients Storing Chatlog and Deleting Select messages

02/09/25

- read messages and messages client x writes will be stored in client x's chatlog

02/09/25

- debating whether to use plaintext or something else - if we write to log file we should be able to see everything (update: yes on .txt)

- if client x deletes a message from client y, it does not show up for client y as a deletion (so only reciever can delete messages)

chatlog

[timestamp] - alice to bob: "hello!"

[timestamp] - bob to alice: "hi!"

[timestamp] - bob to alice: "good morning"

- delete messages by parsing through log by message content, then remove that line from plaintext

- once deleted, gone from log

- if queued messages read, they are immediately added to the chatlog

chatlog

[timestamp] - alice to bob: "hello!"

[timestamp] - bob to alice: "good morning"

- if an account has unread messages, prompt user if they want to read them before deletion. if no, server deletes them

- Should the server log everyone's full chatlog? or only unreads?

Server Stores

→ dict of all registered usernames & hashed passwords (enforce uniqueness in usernames & passwords - no hash collisions!)

→ unread messages to offline clients

→ full chatlogs (with deletions)

→ keeps track of who is messaging who. its in our interest to make this as simple as we can

keep client minimal: server should be universal source of truth. Clients shouldn't store persistent data and should only talk to the server and return values for the GUI to display.

Misc

→ messages in real time are displayed in the terminal when recieved ✓

→ do we want to prompt the client every time and ask if they want to see the message? Could get annoying

02/10/15: Working on GUI. I've never done GUI stuff before so I'm nervous. (Reading up on Tkinter & writing it today probably)

→ Update: This was the most annoying part ever. I shouldn't have done this at the end.

Comparison Between JSON Protocol and our Wire Protocol

02/11/25 - finalizing 02/12/25

→ I wrote a script that simulates client/server communications using our wire protocol on test sockets! (adding it to testing suite)

→ Results were interesting:

```
Test Summary
--------------------
4 tests run in total.
All tests passed!

Detailed Byte Size Totals (Application Payload Only):

Custom (Plain) Mode:
  Sent:     22 bytes
  Received: 22 bytes
  Total:    44 bytes

JSON Mode:
  Sent:     41 bytes
  Received: 49 bytes
  Total:    90 bytes
```

→ For both sending and recieving data, it looks like we're outperforming JSON by close to 50%. Since our custom wire protocol only requires sending two pieces of data (recipient, message) for sending, and only the message contents themselves upon recieving, we're already sending over a lot less raw content than the JSON approach.

→ Also, JSON requires serializing extra characters like braces and dict separation info. We are literally just sending over strings with occaisonal special characters, such as sending recipient and message info.

→ Critically, data isn't parsed into a dict until after it hits the server. This means we're storing heavy info chunks, but sending the bare minimum over the server.

→ We could even compress our protocol further using zlib, which would make us even more compact.

→ TCP/IP overhead is about the same for both methods. ~ +40 bytes.

(code has full details - nothing unexpected with these results, which is great!)